https://doi.org/10.1007/sxxxxx-yyy-zzzz-1

RESEARCH ARTICLE

Registered Attribute-Based Encryption with Reliable Outsourced Decryption Based on Blockchain

Dongliang CAI[1], Liang ZHANG[2,4], Borui CHEN[1], Haibin KAN[1,3]✉

1. College of Computer Science and Artificial Intelligence, Fudan University, Shanghai 200433, China
2. Department of Industrial Engineering and Decision Analytics, Hong Kong University of Science and Technology, Hong Kong
3. Shanghai Institute for Mathematics and Interdisciplinary Sciences, Shanghai 200433, China
4. School of Cyberspace Security (School of Cryptology), Hainan University, Haikou, 570228, China

Received month dd, yyyy; accepted month dd, yyyy

E-mail: hbkan@fudan.edu.cn.

© Higher Education Press 2025

arXiv:2412.08957v3 [cs.CR] 4 Jul 2025

Abstract

Decentralized data sovereignty and secure data exchange are regarded as foundational pillars of the new era. Attribute-based encryption (ABE) is a promising solution that enables fine-grained access control in data sharing. Recently, Hohenberger et al. (Eurocrypt 2023) introduced registered ABE (RABE) to eliminate trusted authority and gain decentralization. Users generate their own public and secret keys and then register their keys and attributes with a transparent key curator. However, RABE still suffers from heavy decryption overhead. A natural approach to address this issue is to outsource decryption to a decryption cloud server (DCS). In this work, we propose the first auditable RABE scheme with reliable outsourced decryption (ORABE) based on blockchain. First, we achieve verifiability of transform ciphertext via a verifiable tag mechanism. Then, the exemptibility, which ensures that the DCS escapes false accusations, is guaranteed by zero knowledge fraud proof under the optimistic assumption. Additionally, our system achieves fairness and auditability to protect the interests of all parties through blockchain. Finally, we give concrete security and theoretical analysis and evaluate our scheme on Ethereum to demonstrate feasibility and efficiency.

Key words

Registered Attribute-Based Encryption; Outsourced Decryption; Blockchain; Non-Interactive Zero Knowledge Proof.

1 Introduction

As the digital world evolves, the metaverse is gaining traction [1], creating new opportunities and challenges for data sharing. The metaverse requires a decentralized and user-centric data sharing framework. Ciphertext-policy attribute-based encryption (CP-ABE) [2] provides a promising primitive to achieve fine-grained access control for data sharing by embedding access structure directly into the encrypted data and associating attributes with decryption keys.

CP-ABE relies on a trusted authority that maintains a master secret key to issue decryption keys [2], limiting its wide adoption in a decentralized environment. If the central authority is compromised, the adversary would be able to decrypt all ciphertexts within the system. The proposal of decentralized ABE still relies on multiple authorities [3]. Therefore, Hohenberger et al. [4] proposed a registered ABE (RABE) scheme without an authority. In RABE, users generate their secret and public keys and register their keys and attributes with a key curator. The curator is transparent, keeps no secrets, and its malicious behavior can always be detected. Subsequently, several constructions of RABE were proposed [5–10] to enhance the original scheme. The pairing-based constructions are more efficient, but many of them [4, 7, 8] have a large and impractical common reference string (CRS). Garg et al. [5] reduced the size of CRS with progression-free sets, which makes the scheme more practical. Considering real-world applications, Weng et al. [9] reduced the decryption overhead for ciphertexts with the same sub-policy in RABE, while Li et al. [10] proposed a revocable RABE scheme to enhance flexibility.

However, ABE schemes [2, 4, 5, 11, 12] suffer from high decryption overhead, which poses a significant challenge for lightweight devices. To address this issue, Green et al. [13] proposed an outsourced decryption scheme for attribute-based encryption (OABE). OABE reduces the computational overhead for users without revealing information about the original data. In their scheme, the decryption cloud server (DCS) transforms the ciphertext into an ElGamal ciphertext [14], which the user then decrypts to retrieve the plaintext. Since the DCS may not perform the computation honestly, many verifiable OABE schemes have been proposed [15–19]. Xu et al. [20] and Li et al. [21] extended verifiability to ensure that both authorized and unauthorized users can verify the correctness of the outsourced decryption, which is called full verifiability in [21]. Nevertheless, many previous schemes fail to guarantee exemptibility and fairness. The exemptibility property ensures

that the DCS is not falsely accused if it returns a correct transform ciphertext. The fairness property ensures that the DCS gets paid if and only if it returns a correct result. To tackle this issue, Cui et al. [22] introduced a payable OABE scheme based on blockchain [23]. Due to the lack of support for pairing computation in Ethereum [24], the OABE scheme proposed by Ge et al. [25] cannot be directly integrated with Ethereum platform.

To the best of our knowledge, no outsourced decryption scheme has been proposed for RABE. Moreover, there is a lack of a reliable and efficient outsourced decryption scheme for ABE. This gap motivates our work. In this paper, we develop the first reliable and efficient outsourced decryption solution for the RABE scheme proposed by Garg et al. [5] based on Ethereum blockchain. Our contributions of this paper are fourfold:

- We propose the first auditable RABE scheme with reliable outsourced decryption (ORABE) based on blockchain, which reduces the user's decryption overhead to only one group exponentiation operation without requiring pairing operations.
- We achieve verifiability of transform ciphertext by a verifiable tag mechanism [17] and ensure exemptibility via non-interactive zero knowledge (NIZK) proof under the optimistic assumption. Moreover, auditability and fairness are ensured based on decentralized blockchain and the payment can be processed by cryptocurrency.
- We propose a decentralized and auditable data sharing framework without a trusted authority. The reliable outsourced decryption makes it friendly for user-centric lightweight devices.
- We give a concrete security analysis of the proposed scheme and implement it on Ethereum to evaluate its feasibility and performance.

2 Related Work

2.1 Outsourced Decryption of Attribute-Based Encryption

ABE was first introduced as fuzzy identity-based encryption by Sahai and Waters in [26]. In CP-ABE, the user can decrypt only if the set of attributes satisfies the access policy of ciphertext. However, the high decryption overhead in ABE greatly hinders its real-world adoption. To mitigate this issue, Green et al. [13] proposed an OABE scheme, which outsources the majority of the decryption to a third party without revealing the plaintext. Consequently, Lai et al. [15] extended the OABE model to achieve verifiability of transform ciphertext. Their approach appends a ciphertext of a random message along with a tag to the original ciphertext, which doubles both the ciphertext size and the transformation cost. Qin et al. [17] reduced the ciphertext size and achieved faster decryption than the scheme proposed in [15]. Lin et al. [18] proposed a more efficient and generic construction of ABE with verifiable outsourced decryption based on an attribute-based key encapsulation mechanism and a symmetric-key encryption scheme. Li et al. [21] extended verifiability to full verifiability, ensuring that both authorized and unauthorized users can verify, which encrypts the plaintext with two access policies for authorized and unauthorized users, respectively. Furthermore, Li et al. [16] proposed an ABE scheme with outsourced encryption and decryption, but it only supports the threshold access policy. Following their work, Ma et al. [27] extended it to accommodate any monotonic access policy and established a stronger security framework compared to [15]. However, the exemptibility they claimed is not achieved since the user may reveal an incorrect private key and accuse the DCS falsely. None of the aforementioned schemes can achieve both exemptibility and fairness. To tackle this issue, Cui et al. [22] introduced a payable OABE scheme that leverages blockchain technology to achieve exemptibility and fairness. However, it still introduces redundant information. Ge et al. [25] eliminated redundant information and mitigated the heavy computation problem of smart contracts through decomposition, but their solution still suffers from high gas consumption and performance limitation. Furthermore, due to the lack of support for pairing computation in native Ethereum [24], this scheme is incompatible with Ethereum platform.

2.2 Registered Attribute-Based Encryption

RABE [4] was introduced recently as a significant extension of ABE, addressing the key escrow problem by replacing the trusted authority that maintains a master secret key to issue attribute keys in traditional ABE with a transparent key curator. In RABE system, users select their public and secret keys independently, and the key curator aggregates users' key-attribute pairs into a master public key. A user encrypts a message with an access policy using the master public key to generate a ciphertext. Any user whose attributes satisfy the access policy in the ciphertext can decrypt it using a helper decryption key and their own secret key. Hohenberger et al. [4] introduced the concept of RABE and provided a construction based on composite order group. Subsequently, several constructions of RABE [5–8, 28–30] were proposed to enhance the original scheme. Some constructions [5, 6, 28–30] require non-black-box use of cryptography, whereas pairing-based constructions [5, 7, 8, 30] do not. The pairing-based constructions are more efficient, but many of them [4, 7, 8, 30] have a large CRS size. Garg et al. [5] reduced it to $L^{1+o(1)}$ (L denotes the number of users) with progression-free sets, thereby making the scheme more practical. Many works explored the improvements of RABE in practical applications recently. Weng et al. [9] presented an efficient RABE scheme for ciphertexts with the same sub-policy in cloud storage. The experimental result shows decryption efficiency is significantly improved with a minimal storage trade-off. Motivated by the revocable ABE scheme [31], Li et al. [10] proposed a revocable RABE scheme with user deregistration to improve flexibility. However, existing pairing-based constructions still suffer from high decryption overhead, which is unfriendly for user-centric lightweight devices. Therefore, we build upon construction [5] and develop a reliable outsourced decryption scheme aimed at minimizing the decryption burden for users.

3 Preliminaries

3.1 Bilinear Map

Let $\mathbb{G}, \mathbb{G}_T$ be two cyclic groups of prime order p, e be the bilinear map $\mathbb{G} \times \mathbb{G} \to \mathbb{G}_T$, and g be the generator of $\mathbb{G}$. The bilinear map has three properties:

- *Bilinearity*: $e(g^a, g^b) = e(g, g)^{ab}$ holds for all $a, b \in \mathbb{Z}_p^*$.

- *Non-degeneracy*: $e(g, g) \neq 1$.
- *Computability*: $e(g, g)$ can be computed in polynomial time.

3.2 Outsourced Decryption CP-ABE

Definition 1. (**Outsourced Decryption CP-ABE** [13]). An outsourced decryption CP-ABE scheme is a tuple of efficient algorithms $\prod_{OABE} = (\textbf{Setup}, \textbf{KeyGen}, \textbf{Encrypt}, \textbf{TKGen}, \textbf{Transform}, \textbf{Decrypt}_{user})$ as follows.

- $\textbf{Setup}(\lambda, \mathcal{U}) \rightarrow PK, MSK$. The algorithm takes a security parameter λ and a universe description $\mathcal{U}$ as inputs. It outputs a public key PK and a master key MSK.
- $\textbf{KeyGen}(MSK, S) \rightarrow SK$. The algorithm takes the master key MSK and a set of attributes S as inputs. It outputs a secret key SK.
- $\textbf{Encrypt}(m, PK, (M, \rho)) \rightarrow CT$. The algorithm takes a message m, the public key PK, and a linear secret sharing scheme (LSSS) access structure (M, ρ) as inputs. It outputs a ciphertext CT.
- $\textbf{TKGen}(SK) \rightarrow TK, z$. The algorithm takes the secret key SK as input. It chooses a random value $z \in \mathbb{Z}_p^*$ and outputs a transform key TK and a retrieve key z.
- $\textbf{Transform}(CT, TK) \rightarrow CT'$. The algorithm takes the transform key TK and the ciphertext CT as inputs. If the transform key does not satisfy the access structure, the algorithm outputs $\perp$, else it outputs a transform ciphertext CT'.
- $\textbf{Decrypt}_{user}(CT', z) \rightarrow m$. The algorithm takes the transform ciphertext CT' and the retrieve key z as inputs. It outputs a message m.

3.3 Registered Attribute-Based Encryption

Definition 2. (**Registered Attribute-Based Encryption** [4]). A RABE scheme with security parameter λ, attribute universe $\mathcal{U}$, and policy space $\mathcal{P}$ is a tuple of algorithms $\prod_{RABE} = (\textbf{Setup}, \textbf{KeyGen}, \textbf{RegPK}, \textbf{Encrypt}, \textbf{Update}, \textbf{Decrypt})$:

- $\textbf{Setup}(1^\lambda, 1^{|\mathcal{U}|}, 1^L) \rightarrow crs$. The algorithm takes the security parameter λ, the attribute universe $\mathcal{U}$, and the number of users L as inputs and generates a common reference string crs as output.
- $\textbf{KeyGen}(crs, aux) \rightarrow (pk, sk)$. The algorithm takes the crs and a state aux as inputs. It outputs a public/secret key pair (pk, sk).
- $\textbf{RegPK}(crs, aux, pk, S_{pk}) \rightarrow mpk, aux'$. The algorithm takes the crs, a state aux, and a public key pk with corresponding attributes $S_{pk} \subseteq \mathcal{U}$ as inputs. It outputs the master public key mpk and an updated state aux'.
- $\textbf{Encrypt}(mpk, P, m) \rightarrow ct$. The algorithm takes the master public key mpk, an access policy $P \in \mathcal{P}$, and a message m as inputs. It outputs a ciphertext ct.
- $\textbf{Update}(crs, aux, pk) \rightarrow hsk$. The algorithm takes the crs, a state aux, and the public key pk as inputs. It outputs a helper decryption key hsk.
- $\textbf{Decrypt}(sk, hsk, ct) \rightarrow m \cup \{\perp, \textbf{GetUpdate}\}$. The algorithm takes the secret key sk, the helper decryption key hsk, and the ciphertext ct as inputs. It either outputs a message m, a decryption failure symbol $\perp$, or a flag **GetUpdate** that indicates an updated hsk is required.

3.4 Sigma Protocol for Discrete Logarithm Equality

$\mathcal{R}_{DLEQ} = \{(g, G, t, T), s | G = g^s \wedge T = t^s\}$ is the discrete logarithm equality (DLEQ) relation. A sigma protocol $\prod_{DLEQ}$ for DLEQ is defined as follows.

- The prover chooses $r \xleftarrow{R} \mathbb{Z}_p$ and sends $C_1 = g^r$ and $C_2 = t^r$ to verifier.
- The verifier samples $c \xleftarrow{R} \mathbb{Z}_p$ and sends it to the prover.
- The prover answers with $z = r + cs$.
- The verifier accepts if $g^z = C_1 G^c$ and $t^z = C_2 T^c$.

The property of $\prod_{DLEQ}$ sigma protocol:

- **Completeness.** For any $(x, w) \in \mathcal{R}$, the prover convinces the verifier to accept with probability 1.
- **Special Soundness.** Given any x and two accepting conversations on input x: (C_1, C_2, c, z), (C_1, C_2, c', z'), where $c \neq c'$, one can efficiently compute w such that $(x, w) \in \mathcal{R}$.
- **Special Honest-Verifier Zero Knowledge.** There exists a polynomial time simulator that, given x and a random challenge c, can output an accepting conversation of the form (C_1, C_2, c, z) with the same probability distribution as conversations between the honest prover and verifier on input x.

The protocol can be made non-interactive using the Fiat-Shamir transformation [32] with a random oracle H. $\prod_{DLEQ-NIZK}$ is defined as follows:

- $\textbf{Prove}(x, w) \rightarrow \pi$. On input $x = (g, G, t, T)$ and $w = s$, the algorithm samples $r \xleftarrow{R} \mathbb{Z}_p$ and computes:

$$C_1 = g^r, C_2 = t^r, c = H(G, T, C_1, C_2), z = r + cs.$$

  Finally, it outputs a proof $\pi = (C_1, C_2, c, z)$.
- $\textbf{Verify}(x, \pi) \rightarrow 0/1$. On input $x = (g, G, t, T)$ and proof $\pi = (C_1, C_2, c, z)$, the algorithm checks:

$$c \stackrel{?}{=} H(G, T, C_1, C_2), g^z \stackrel{?}{=} C_1 G^c, t^z \stackrel{?}{=} C_2 T^c.$$

  It outputs 1 if all equations hold, indicating that the proof is valid, and 0 otherwise.

3.5 Ethereum Blockchain

Ethereum [24] is a decentralized and open-source blockchain platform that incorporates smart contract functionality. One of the key characteristics of Ethereum is decentralization. Ethereum network is not controlled by a central authority and the decentralized consensus mechanism allows Ethereum to provide a trustless environment, ensuring security and immutability. Smart contracts are self-executing programs with the terms of the agreement directly written into code. They automatically execute the predefined actions without the need for a trusted intermediary. Every transaction or computation on Ethereum requires a certain amount of gas, which serves as a fee to incentive

validators. The gas mechanism ensures that resources on the network are used efficiently and prevents abuse by requiring users to pay proportionally to the complexity of their operations.

4 System Model and Threat Model

4.1 System Model

Our system consists of seven entities: Key Curator (KC), InterPlanetary File System (IPFS), Blockchain, Data Owners (DOs), Data Users (DUs), Decryption Cloud Server (DCS) Network, and Public Verifiers (PVs). These seven entities are defined as follows:

- The KC is responsible for user registration and generates the master public key and auxiliary data for users.
- IPFS is a distributed file system used to store ciphertexts in our system without a central server.
- Blockchain serves as a decentralized and immutable ledger that records decryption tasks, fraud proofs, and facilitates smart contract execution.
- DOs encrypt data with specified access structure and upload the ciphertext to IPFS.
- DUs generate their own keys to register with KC. They publish outsourced decryption tasks on blockchain and finish the final decryption locally. Besides, they can generate a fraud proof to accuse dishonest DCS.
- The DCS network consists of many servers that compete for tasks. These servers compute and submit task result to blockchain.
- PVs verify the fraud proof and publish the verification result on blockchain.

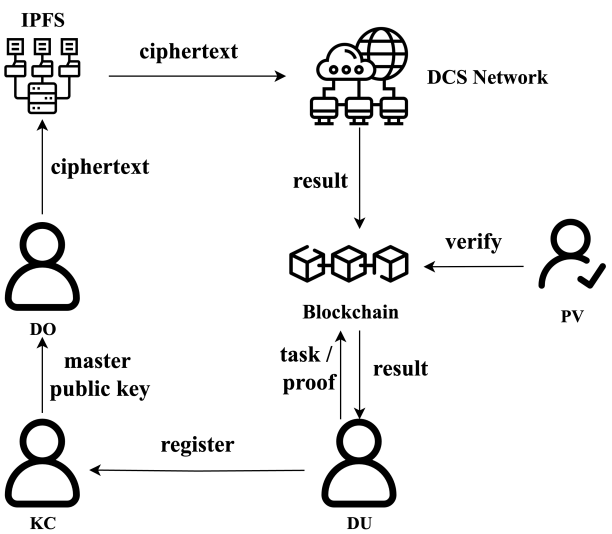


Fig. 1 Brief overview of system entities.

Fig 1 illustrates the workflow among all entities in the system model. In brief, the KC stores system public information and is required in user registration. IPFS serves as the data repository to store RABE ciphertext uploaded by the DO. The DU, wishing to decrypt the ciphertext, submits a decryption task on the blockchain and deposits tokens. Any DCS can perform the outsourced decryption and submit the result to the blockchain. In the happy case, the DU retrieves the DO's data and the DCS receives the tokens. In the dispute phase, the deposited tokens are returned to the DU if the fraud proof passes verification by the PV.

4.2 Threat Model

In our system, DOs and IPFS are fully trusted. The KC and PVs are semi-honest and transparent, which means they keep no secrets and perform computations honestly. DCSs are malicious, as they may deliberately submit incorrect results to obtain task rewards. Additionally, since DUs may attempt to submit invalid fraud proof to avoid payment, we also assume they are malicious. The optimistic assumption supposes that all parties are honest unless proven otherwise.

4.3 Security Requirements

- **Static Security.** The static security ensures that data is confidential to unauthorized users.
  Let $\prod_{OSRABE}$ = (**Setup**, **KeyGen**, **RegPK**, **Encrypt**, **Transform**, **Decrypt**$_{user}$) be a slotted outsourced decryption RABE scheme with security parameter λ. We define the following game $Game_{sec}$ between a challenger C and an adversary $\mathcal{A}$:
  - **Setup**: The challenger C generates crs and sends it to $\mathcal{A}$. Let the master public key $mpk = \perp$ and the counter $ctr = 0$.
  - **Query**: The challenger C responds to the adversary's key-generation queries on a non-corrupted slot index $i \in N$ with a set of attributes S. C computes the counter $ctr = ctr + 1$ and samples $pk_{ctr}, sk_{ctr} \leftarrow$ **KeyGen**(crs, ctr). C replies to $\mathcal{A}$ with ctr and pk_{ctr}. Note that in the static security game, the adversary is not allowed to make any corruption queries.
  - **Challenge**: The adversary $\mathcal{A}$ selects two messages μ_0^*, μ_1^* in message space and an access policy P^* in policy space. The challenger C calls **RegPK** to generate mpk and hsk_i. Then, C randomly selects $b \in \{0, 1\}$ and replies the challenge ciphertext:
    $$ct^* \leftarrow \textbf{Encrypt}(mpk, P^*, \mu_b^*).$$
  - **Output**: $\mathcal{A}$ outputs a bit $b' \in \{0, 1\}$.

  The advantage of the adversary in $Game_{sec}$ is defined as: $\mathbf{Pr}[b' = b] - \frac{1}{2}$.

  Definition 3. An OSRABE scheme is secure if all probabilistic polynomial time (PPT) adversaries have at most a negligible advantage in the above game:
  $$\mathbf{Pr}[b' = b] - \frac{1}{2} \leq negl(\lambda)$$
- **Verifiability of outsourced decryption.** The verifiability guarantees that the correctness of transform ciphertext can be efficiently checked by authorized users.
- **Exemptibility of outsourced decryption.** The exemptibility guarantees that users cannot falsely accuse the DCS of returning the invalid transform ciphertext.

- **Fairness.** The fairness property in our work guarantees that the DCS gets paid if and only if it has returned a correct result, while the DU must pay for the correct result and is entitled to challenge an incorrect one through fraud proof. Fairness incentivizes each party to perform the protocol honestly.
- **Auditability.** The auditability guarantees the behaviors of the transparent and semi-honest participants (i.e., KC and PV) can be publicly audited.

5 Construction

5.1 Verifiable Outsourced Decryption Slotted RABE

A slotted RABE (SRABE) scheme keeps a fixed number of slots and does not support dynamic registrations. Based on scheme [5], we propose a SRABE with verifiable outsourced decryption (OSRABE) scheme, which is defined as follows:

- **Setup**$(1^\lambda, 1^{|\mathcal{U}|}, 1^L) \rightarrow crs$. On input the security parameter λ, the attribute universe $\mathcal{U}$, and the number of slots L, the algorithm samples a prime order pairing group: $(\mathbb{G}, \mathbb{G}_T, g, e)$. Let $\mathcal{D} = \{d_i\}_{i\in[1,L]}$ be a progression-free and double-free set. Next, it defines:

$$f(i,j) := d_i + d_j \quad \text{and} \quad \mathcal{E} = \{f(i,j) | i,j \in [1,L] : i \neq j\}.$$

For $i \in [1,L]$, it samples $a, b, \gamma_i \xleftarrow{R} \mathbb{Z}_p$ and sets $t_i = a^{d_i}, \alpha = -a^{3\cdot d_{max}}$. Then, it computes:

$$h = \prod_{i\in[1,L]} g^{a^{3\cdot d_{max}-d_i}}, A_i = g^{t_i}, B_i = g^{\alpha} h^{t_i}, P_i = g^{\gamma_i}, U_i = g^{bt_i}$$

let $Z = e(g,g)^\alpha$ and $W_z = g^{ba^z}$ for each $z \in \mathcal{E}$. Finally, it sets $pp = (\mathbb{G}, \mathbb{G}_T, g, e, Z, h)$ and outputs the common reference string crs :

$$crs = (pp, \{(A_i, B_i, P_i, U_i)\}_{i\in[1,L]}, \{W_z\}_{z\in\mathcal{E}}).$$

- **KeyGen**$(crs, i) \rightarrow pk_i, sk_i$. On input the crs and a slot index i, the algorithm first selects $r_i \xleftarrow{R} \mathbb{Z}_p$ and then computes:

$$T_i = g^{r_i}, Q_i = P_i^{r_i}, V_{j,i} = A_j^{r_i} (j \neq i).$$

Finally, it outputs the public key pk_i and the secret key sk_i :

$$pk_i = (T_i, Q_i, \{V_{j,i}\}_{j\neq i}) \ , \ sk_i = r_i.$$

- **RegPK**$(crs, \{(pk_i, S_i)\}_{i\in[1,L]}) \rightarrow mpk, \{hsk_i\}_{i\in[1,L]}$. On input the crs and a list of public key-attribute pairs, the algorithm computes $\hat{T}$ and $\hat{V}_i$ for each $i \in [1,L]$:

$$\hat{T} = \prod_{j\in[1,L]} T_j \quad , \quad \hat{V}_i = \prod_{j\neq i} V_{i,j}.$$

For each attribute $w \in \mathcal{U}$, it computes $\hat{U_w}$ and $\hat{W_{i,w}}$ for each $i \in [1,L]$:

$$\hat{U}_w = \prod_{j\in[1,L]:w\notin S_j} U_j \quad , \quad \hat{W}_{i,w} = \prod_{j\neq i:w\notin S_j} W_{f(i,j)}.$$

Finally, it selects three hash functions H_0, H_1, H_2 and outputs the master public key mpk and the helper decryption key hsk_i for each $i \in [1,L]$:

$$mpk = (pp, \hat{T}, \{\hat{U}_w\}_{w\in\mathcal{U}}, H_0, H_1, H_2),$$

$$\{hsk_i = (i, S_i, A_i, B_i, \hat{V}_i, \{\hat{W}_{i,w}\}_{w\in\mathcal{U}})\}_{i\in[1,L]}.$$

- **Encrypt**$(mpk, (\mathbf{M}, \rho), m) \rightarrow ct$. On input the master public key mpk, the access structure $(\mathbf{M}, \rho)$ where $\mathbf{M}$ is a $\beta \times n$ matrix and ρ maps $\mathbf{M}_i$ (i-th row of $\mathbf{M}$) to an attribute $\rho(i)$, and a message m, the algorithm first samples $\mu \in \mathbb{G}_T$ and computes t_2 as symmetric key. **Enc** is a symmetric encryption algorithm. Then, it samples $s, v_2, ..., v_n \xleftarrow{R} \mathbb{Z}_p$ and $h_1, h_2 \xleftarrow{R} \mathbb{G}$ such that $h = h_1 h_2$ and sets $v = [s, v_2, ..., v_n]^T$. For each $k \in [1,\beta]$, it samples $s_k \xleftarrow{R} \mathbb{Z}_p$ and constructs the ciphertext as follows:

$$t_1 = H_0(\mu), t_2 = H_1(\mu), C = \mathbf{Enc}_{t_2}(m), tag = H_2(t_1||C)$$

$$C_1 = \mu \cdot Z^s, C_2 = g^s$$

$$C_{3,k} = h_2^{m_k^T v} \hat{U}_{\rho(k)}^{-s_k}, C_{4,k} = g^{s_k}, C_5 = (h_1 \hat{T}^{-1})^s$$

Finally, it outputs the ciphertext ct :

$$ct = ((\mathbf{M}, \rho), C, C_1, C_2, \{C_{3,k}, C_{4,k}\}_{k\in[1,\beta]}, C_5, tag).$$

- **Transform**$(hsk, ct) \rightarrow ct'$. On input the ciphertext ct and the helper decryption key hsk, the algorithm outputs $\perp$ if the set of attributes in hsk does not satisfy the access structure in ct. Otherwise, it sets $I = \{j : \rho(j) \in S_i\}$ and computes the set $\{\omega_j | \omega_j \in \mathbb{Z}_p\}$ such that $\sum_{j\in I} \omega_j \mathbf{M}_j = (1, 0, ..., 0)$. Next, it computes:

$$C_1' = \frac{C_1}{e(C_2, B_i)} \cdot e(C_5, A_i) \cdot e(C_2, \hat{V}_i) \cdot \prod_{j\in I} \left(e(C_{3,j}, A_i) \cdot e(C_{4,j}, \hat{W}_{i,\rho(j)})\right)^{\omega_j}$$

$$C_2' = e(C_2, A_i)$$

Finally, it outputs the transform ciphertext $ct' = (C_1', C_2')$.

- **Decrypt**$_{user}(sk_i, ct', ct) \rightarrow m/\perp$. On input the original ciphertext ct, the transform ciphertext ct', and the secret key $sk_i = r_i$, the algorithm computes:

$$\mu' = C_1' \cdot C_2'^{r_i}.$$

Then, it checks:

$$tag \stackrel{?}{=} H_2(H_0(\mu')||C).$$

If the equation does not hold, the algorithm outputs $\perp$, which indicates the transform ciphertext ct' is invalid. Otherwise, it uses the symmetric decryption algorithm **Dec** to recover message $m = \mathbf{Dec}_{H_1(\mu')}(C)$ and outputs m.

5.2 Verifiable Outsourced Decryption RABE

Hohenberger et al. [4] constructed RABE using multiple SRABE instances. We use the same transformation to construct our ORABE scheme based on the OSRABE scheme described in Section 5.1. Our verifiable ORABE is defined as follows:

- **Setup**$(1^\lambda, 1^{|\mathcal{U}|}, 1^L) \rightarrow crs$. On input the security parameter λ, the attribute universe $\mathcal{U}$, and the limit number of users $L = 2^l$, the algorithm initializes $l+1$ OSRABE instances and generates $crs_k \leftarrow$ **OSRABE.Setup**$(1^\lambda, 1^{\mathcal{U}}, 1^{2^k})$ for each $k \in [0, l]$. Finally, it outputs:

$$crs = (crs_0, ..., crs_l).$$

- **KeyGen**$(crs, aux) \rightarrow pk, sk$. On input the crs and the auxiliary data $aux = (ctr, Dict_1, Dict_2, mpk)$, the algorithm generates public/secret key for each OSRABE instance. For each $k \in [0, l]$, it computes slot index $i_k = (ctr \bmod 2^k) + 1$ and generates $(pk_k, sk_k) \leftarrow$ **OSRABE.KeyGen**(crs_k, i_k). Finally, it outputs:

$$pk = (ctr, pk_0, ..., pk_l),$$

$$sk = (ctr, sk_0, ..., sk_l).$$

- **RegPK**$(crs, aux, pk, S_{pk}) \rightarrow mpk', aux'$. On input the $crs = (crs_0, ..., crs_l)$, the auxiliary data $aux = (ctr_{aux}, Dict_1, Dict_2, mpk)$, where $mpk = (ctr_{aux}, mpk_0, ..., mpk_l)$, public key $pk = (ctr_{pk}, pk_0, ..., pk_l)$, and associated attribute set S_{pk}, the algorithm executes as follows:
  - For each $k \in [0, l]$, it computes $i_k = (ctr_{aux} \bmod 2^k) + 1$ for the k-th scheme. It terminates if $ctr_{aux} \neq ctr_{pk}$.
  - For each $k \in [0, l]$, it sets $Dict_1[k, i_k] = (pk, S_{pk})$. If $i_k = 2^k$, it generates $(mpk'_k, \{hsk'_{k,i}\}_{i \in [1, 2^k]}) \leftarrow$ **OSRABE.RegPK**$(crs_k, Dict_1[k])$ and then updates $Dict_2[ctr + i + 1 - 2^k, k] = hsk'_{k,i}$ for each $i \in [1, 2^k]$.

  Finally, it outputs:

$$mpk' = (ctr_{aux} + 1, mpk'_0, ..., mpk'_l),$$

$$aux' = (ctr_{aux} + 1, Dict_1, Dict_2, mpk').$$

- **Encrypt**$(mpk, (\mathbf{M}, \rho), m) \rightarrow ct$. On input the master public key $mpk = (ctr, mpk_0, ..., mpk_l)$, the access structure $(\mathbf{M}, \rho)$ and a message m, for each $k \in [0, l]$, the algorithm computes:

$$ct_k \leftarrow \textbf{OSRABE.Encrypt}(mpk_k, (\mathbf{M}, \rho), m)$$

  If $mpk_k = \perp$, then it sets $ct_k = \perp$. It outputs:

$$ct = (ctr, ct_0, ..., ct_l).$$

- **Transform**$(hsk_k, ct_k) \rightarrow ct'$. On input the ciphertext ct_k and the helper decryption key hsk_k, the algorithm computes:

$$ct' \leftarrow \textbf{OSRABE.Transform}(hsk_k, ct_k).$$

  It outputs ct'.

- **Update**$(crs, aux, pk) \rightarrow hsk$: On input the common reference string $crs = (crs_0, ..., crs_l)$, the auxiliary data $aux = (ctr_{aux}, Dict_1, Dict_2, mpk)$, and the public key $pk = (ctr_{pk}, pk_0, ..., pk_l)$. If $ctr_{pk} < ctr_{aux}$, it sets $hsk_k = Dict_2[ctr_{pk} + 1, k]$ for each $k \in [0, l]$; otherwise, it outputs $\perp$. Finally, it outputs:

$$hsk = (hsk_0, ..., hsk_l).$$

- **Decrypt**$_{user}(sk_k, ct', ct_k) \rightarrow m/\perp$. On input the transform ciphertext ct', secret key sk_k, and original ciphertext ct_k, the algorithm verifies ct' and computes m:

$$m/\perp \leftarrow \textbf{OSRABE.Decrypt}_{user}(sk_k, ct', ct_k).$$

  Finally, it outputs m if the verification of ct' is successful. Otherwise, it outputs $\perp$, which indicates the transform ciphertext ct' is invalid.

5.3 Exemptibility via NIZK Fraud Proof

The fraud proof mechanism enables challengers to dispute the previous result by proving its incorrectness. If the decrypt algorithm outputs $\perp$, it indicates that the transform ciphertext is invalid. In this case, the DU could generate a valid fraud proof to accuse the DCS. Our fraud proof is used to prove the transform ciphertext is not correct. This not only provides a mechanism for the DU to demonstrate malicious behavior of the DCS but also ensures that the DCS cannot be maliciously accused, which achieves exemptibility of the outsourced decryption. More specifically, we establish an accountability mechanism based on the following two critical points:

- The DU can claim malicious behavior of the DCS through fraud proof, which proves the transform ciphertext with the secret key cannot recover the original plaintext. This mechanism protects the DU from potential economic losses.
- It also guarantees exemptibility by preventing false accusations through the NIZK proof that validates decryption failures without revealing secret keys. This method effectively resolves the exemptibility vulnerability identified in [27], where malicious users could potentially fabricate accusations by revealing invalid private keys. We achieve exemptibility through DLEQ NIZK protocol rather than key exposure, thereby maintaining secret key confidentiality while ensuring fairness.

The fraud proof mechanism based on the DLEQ NIZK proof is designed as follows:

- **Prove**$(sk, mpk, ct') \rightarrow \pi$: On input the master public key mpk, the secret key sk, and the transform ciphertext $ct' = (C'_1, C'_2)$, the algorithm computes the verification key $vk = C_2'^{sk}$ and generates a NIZK proof π' to prove the correctness of vk:

$$\pi' \leftarrow \prod_{\text{DLEQ-NIZK}} .\textbf{prove}(C'_2, vk, g, g^{sk})$$

  Note that C'_2 and $pk = g^{sk}$ are public. Finally, it outputs the fraud proof:

$$\pi = (\pi', vk).$$

Contract **ORABE**

———-(user registration / key management)———-

PublishState: Receive $(ctr, auxHash, pkHash)$ as input, where ctr is ctr_{aux} in the aux and can be regarded as the index of user U, $auxHash$ is hash(aux) and $pkHash$ is hash($\mathbf{pk}_u$). The smart contract records these state data on blockchain.

1) Assert msg.sender is KC
2) state = new KCState(ctr, $auxHash$, $pkHash$)
3) KCstates.append(state)

———————(ciphertext management)———————

PublishTag: Receive $(index, ctTag)$ as input, where $index$ is the index of ciphertext ct in IPFS, and $ctTag$ is tag in ct. The smart contract records the mapping relation between $index$ and $ctTag$.

1) Assert msg.sender is PV
2) Tags[$index$] = $ctTag$

———————(fraud proof management)———————

PublishFraudProof: Receive (id_{task}, π) as input, where π is the fraud proof.

1) Assert Tasks[$taskId$].status == $Submitted$
2) Assert Tasks[$taskId$].publisher == msg.sender
3) Assert $T_{now} \leq$ Tasks[$taskId$].challengeDeadline
4) Tasks[$taskId$].fraudproof = π
5) Tasks[$taskId$].status = $Challenged$

PublishVerificationResult: Receive (id_{task}, b) as input, where b is a boolean value that indicates whether the proof is valid($b = 1$) or not($b = 0$).

1) Assert msg.sender is PV
2) Assert Tasks[$taskId$].status == $Challenged$
3) Tasks[$taskId$].proofIsValid = b
4) Tasks[$taskId$].status = $Completed$
5) _**DisposeReward**(id_{task}, b)

———————(task management)———————

CreateTask: Receive $(ctTag, ctr_u)$ as input, where $ctTag$ is tag of the ciphertext to be decrypted, and ctr_u is ctr in the public key $\mathbf{pk}_u$ of DU. Meanwhile, the invoker also send cryptocurrency of amount $\$reward$ as task reward. Function returns id_{task} which is the unique identifier of the created task.

1) Assert $\$reward > 0$
2) id_{task} := hash(msg.sender, $ctTag$, $\$reward$, T_{now})
3) Tasks[id_{task}] = new Task(id: id_{task}, publisher: msg.sender, ctTag: $ctTag$, ctr: ctr_u, reward: $\$reward$, status: $Published$)
4) Return id_{task}

SubmitResult: Receive (id_{task}, ct') as input, where id_{task} is task identifier, and ct' are partially decrypted result.

1) Assert Tasks[$taskId$].status == $Published$
2) Tasks[$taskId$].submitter = msg.sender
3) Tasks[$taskId$].status = $Submitted$
4) Tasks[$taskId$].result = ct'
5) Tasks[$taskId$].challengeDeadline = $T_{now} + T_p$

ClaimReward: Receive (id_{task}) as input.

1) Assert Tasks[$taskId$].status == $Submitted$
2) Assert Tasks[$taskId$].submitter == msg.sender
3) Assert $T_{now} >$ Tasks[$taskId$].challengeDeadline
4) Tasks[$taskId$].status = $Completed$
5) Transfer(msg.sender, Tasks[$taskId$].reward)

———————(auxiliary functions)———————

_**DisposeReward**: Receive (id_{task}, b) as input. This function dispose reward automatically according to b.

1) reward := Tasks[$taskId$].reward
2) If (b == 1):
   Transfer(Tasks[$taskId$].publisher, reward)
3) If (b == 0):
   Transfer(Tasks[$taskId$].submitter, reward)

Fig. 2 Contract design for ORABE.

- **Verify**$(\pi, ct', mpk, tag) \rightarrow 0/1$: On input the fraud proof π, the transform ciphertext ct', the master public key mpk, and the ciphertext tag, the algorithm first checks the correctness of vk:

$$\prod_{\text{DLEQ-NIZK}} .\textbf{Verify}(\pi', (C_2', vk, g, T_i))$$

If the proof is invalid, it outputs 0. Otherwise, it computes:

$$\mu'' = C_1' \cdot vk.$$

Then, it checks:

$$tag \stackrel{?}{=} H_2(H_0(\mu'')||C).$$

If the equation does not hold, it outputs 1 to indicate the fraud proof is valid (i.e., the transform ciphertext is invalid), and 0 otherwise.

Discussion. Unfortunately, this verify algorithm cannot be executed by Ethereum smart contract because it does not support $\mathbb{G}_t$ computation of any elliptic curve now. Therefore, we introduce a semi-honest and transparent public verifier to verify the fraud proof and publish the verification result on blockchain. Ge et al. [25] added precompile contract to native Ethereum to support pairing and $\mathbb{G}_t$ computation on their own blockchain. If we reduce the reliance on native Ethereum security, we can integrate their approach to eliminate the public verifier role and enable the verification of fraud proof through smart contracts directly. In this work, we make a trade-off and prioritize the security of native Ethereum. Note that in our ORABE scheme, the public verifier role can be played by KC, which is semi-honest and transparent.

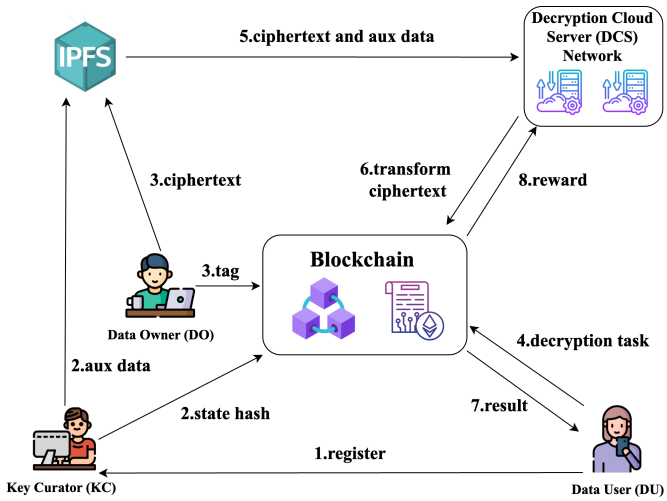


Fig. 3 Overview of happy case.

5.4 Smart Contract

The smart contract design is shown in Fig 2, which manages the whole procedure of user registration, ciphertext tag publication, outsourced decryption, and public verification of fraud proof. Each DU first sends $\mathbf{pk}_u$ and attributes to the KC. On receiving message from each DU, the KC allocates a suitable ctr to the DU and updates the auxiliary data aux. To make sure that the KC's behavior is auditable, the KC should invoke **PublishState** when it finishes registration. The state data consists of ctr, hash($\mathbf{pk}_u$), and hash(aux), which ensures the state transformation can be verified publicly. Ciphertexts are stored in IPFS, and the DO should invoke **PublishTag** after uploading ciphertext. Regarding outsourced decryption, the DU first publishes an outsourced decryption task by invoking **CreateTask** with some tokens as the reward. Then, the DCS performs partial decryption and invokes **SubmitResult** on the blockchain. In the happy case, the DCS could invoke **ClaimReward** after the challenge window. In the dispute phase, the DU can generate a fraud proof and invoke **PublishFraudProof** during the challenge window. The PV verifies the fraud proof and invokes **PublishVerificationResult** on blockchain.

5.5 Use Case

We propose a decentralized data sharing framework based on the auditable and reliable ORABE scheme under the optimistic assumption. The optimistic assumption supposes that all parties are honest unless proven otherwise. The dispute phase and public verifier are needed only when one player crashes or attempts to cheat. Fig 3 illustrates the overview of the happy case without challenge, which consists of the following steps:

1. The DU generates secret key and public key locally and sends public key and attributes to the KC.
2. The KC computes and updates the auxiliary data on IPFS, and publishes new state hash and public key hash on blockchain.
3. The DO encrypts data under a key encapsulation mechanism, uploads ciphertext to IPFS, and publishes ciphertext tag on-chain.
4. The DU publishes an outsourced decryption task with the reward on-chain.
5. The DCS observes the task and downloads the corresponding ciphertext and auxiliary data from IPFS.
6. The DCS computes and submits the transform ciphertext to blockchain.
7. The DU gets the transform ciphertext from blockchain and completes the decryption locally.
8. The DCS claims the task reward from blockchain after the challenge window.

After the final decryption, the DU verifies whether the plaintext matches the ciphertext tag. If the verification fails, as shown in Fig 4, the DU enters the dispute phase, which consists of the following four steps:

1. The DU generates a fraud proof and publishes it on-chain to accuse the DCS within the challenge window.
2. The PV obtains the fraud proof from blockchain.
3. The PV verifies the fraud proof and publishes the verification result on blockchain.
4. The smart contract returns the deposit to the DU if the fraud proof is valid. Otherwise, the DCS receives the payment.

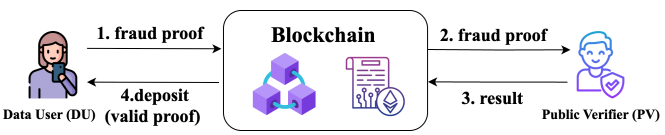


Fig. 4 Overview of dispute case.

6 Security Analysis

Theorem 1. (OSRABE Static Security) Suppose the SRABE scheme of [5] is statically secure, then the proposed OSRABE scheme is also statically secure.

Proof. Suppose there exists a PPT adversary $\mathcal{A}$ that can attack our scheme in the CPA security model with advantage ϵ, then we can build a simulator $\mathcal{B}$ that can break the SRABE construction 4.3 in [5] with advantage ϵ.

Setup. The challenger C generates crs using the setup algorithm. The simulator $\mathcal{B}$ gets crs from C and returns it to the adversary $\mathcal{A}$. Let the master public key $mpk = \perp$ and the counter $ctr = 0$.

Query. The simulator $\mathcal{B}$ answers the adversary $\mathcal{A}$'s key-generation queries on a non-corrupted slot index $i \in N$ with a set of attributes S as follows:

- The simulator $\mathcal{B}$ passes the query to the challenger C.
- C computes the counter $ctr = ctr+1$ and samples $pk_{ctr}, sk_{ctr} \leftarrow \mathbf{KeyGen}(crs, ctr)$. C replies to $\mathcal{B}$ with (ctr, pk_{ctr}).
- $\mathcal{B}$ passes (ctr, pk_{ctr}) to $\mathcal{A}$.

Challenge. $\mathcal{A}$ submits a plaintext pair $(K_0^*, K_1^*) \in \{0,1\}^{2\times k}$ to $\mathcal{B}$, and $\mathcal{B}$ sends them to the challenger C. C generates the challenge ciphertext pair (CT_0^*, CT_1^*) corresponds to (K_0^*, K_1^*) and sends $CT_b^* (b \in \{0,1\})$ to $\mathcal{B}$.

Table 1 Comparison of Pairing-Based RABE Schemes

Schemes	$\|crs\|$	$\|st\|$	$\|mpk\|$	$\|hsk\|$	$\|ct\|$	Decryption Cost	Assumption	Security
[4]	$\|\mathcal{U}\|L^2$	$\|\mathcal{U}\|L^2$	$\|\mathcal{U}\|$	$\|\mathcal{U}\|$	$\|\mathcal{P}\|$	$\|\mathcal{P}\|(\mathbb{P}+\mathbb{E}_T)$	Composite-order	Adaptive
[5]	$L^{(1+o(1))}$	$\|\mathcal{U}\|L$	$\|\mathcal{U}\|$	$\|\mathcal{U}\|$	$\|\mathcal{P}\|$	$\|\mathcal{P}\|(\mathbb{P}+\mathbb{E}_T)$	Prime-Order	Static
[7]	$\|\mathcal{U}\|L^2$	$\|\mathcal{U}\|L^2$	$\|\mathcal{U}\|$	$\|\mathcal{U}\|$	$\|\mathcal{P}\|$	$\|\mathcal{P}\|(\mathbb{P}+\mathbb{E}_T)$	Prime-Order	Adaptive
[8]	L^2	L^2	δ^2	δ^2	$\|\mathcal{P}\|$	$\|\mathcal{P}\|(\mathbb{P}+\mathbb{E}_T)$	Prime-Order	Adaptive
Ours	$L^{(1+o(1))}$	$\|\mathcal{U}\|L$	$\|\mathcal{U}\|$	$\|\mathcal{U}\|$	$\|\mathcal{P}\|$	$\mathbb{E}_T$	Prime-Order	Static

Note: st denotes the auxiliary data, L denotes the number of users, $\mathcal{U}$ is attribute universe, and $\mathcal{P}$ is access policy. In scheme [8], $\delta = max_{i\in[1,L]}|S_i|$ (S_i is a registered attribute set). $\mathbb{P}$ denotes the pairing operation, $\mathbb{E}_T$ denotes the $\mathbb{G}_T$ exponentiation operation.

Guess. $\mathcal{B}$ passes ciphertext CT_b^* to $\mathcal{A}$ and gets the response bit 0 or 1. $\mathcal{B}$ outputs $\mathcal{A}$'s response as its guess.

Note that the simulation of $\mathcal{B}$ is perfect if the above game does not abort and the construction 4.3 of [5] has been proven statically secure. The advantage of the simulator breaks the security of the construction in [5] is the same as the adversary that breaks the CPA security of our scheme. Thus, our proposed scheme 5.1 is statically secure. □

Theorem 2. (ORABE Static Security) Based on Theorem 1, the proposed ORABE scheme is statically secure.

Proof. While [4] only analyzes the security of the RABE scheme transformed from a fully secure SRABE scheme, the security analysis of the RABE scheme transformed from statically-secure SRABE scheme is proposed in [5]. Therefore, our ORABE scheme inherits the statically secure property from the OSRABE scheme. □

Theorem 3. (Verifiability) Assume that H_0^*, H_2^* are collision-resistant hash functions, the proposed ORABE scheme achieves the verifiability.

Proof. The verifiability is broken if DCS can return two different transform ciphertexts to match the original tag. Then, we can get:

$$H_2^*(H_0^*(\mu_1^*)||C) = tag = H_2^*(H_0^*(\mu_2^*)||C), \text{where } \mu_1^* \neq \mu_2^*.$$

If $H_0^*(\mu_1^*) = H_0^*(\mu_2^*)$, then we can break the collision-resistance of H_0^*, else we can break the collision-resistance of H_2^*. □

Theorem 4. (Exemptibility) Suppose that the soundness of protocol $\prod_{DLEQ-NIZK}$ holds, the proposed ORABE scheme achieves the exemptibility.

Proof. The exemptibility is broken if DU can return two different valid verification keys vk_1 and vk_2 in two fraud proofs. Then, we can get:

$$1 \leftarrow \prod_{DLEQ-NIZK}.\mathbf{Verify}(\pi_1, (C_2', vk_1, g, T_i))$$

$$1 \leftarrow \prod_{DLEQ-NIZK}.\mathbf{Verify}(\pi_2, (C_2', vk_2, g, T_i))$$

where $vk_1 \neq vk_2$. Thus we can break the soundness of $\prod_{DLEQ-NIZK}$ protocol. □

Theorem 5. (Fairness) The proposed scheme based on Ethereum block-chain ensures fairness.

Proof. The fairness of our scheme ensures that the DCS gets paid if and only if it returns a correct result. The DU must deposit tokens into the smart contract to submit the outsourced decryption task. If the DCS submits the correct result, it will receive the reward at the end of the challenge window, which is ensured by exemptibility. Otherwise, the malicious behavior will be detected and proved by the DU, which is ensured by verifiability and NIZK proof, and the DCS get nothing but pay for the transaction gas fee for dishonesty. Since our payments are made through Ethereum smart contracts, we also need to rely on Ethereum consensus protocol. Thus the proposed scheme achieves the fairness property as long as the verifiability, exemptibility, and Ethereum consensus protocol are guaranteed. □

Theorem 6. (Auditability) The proposed scheme based on publicly verifiable NIZK proof and Ethereum blockchain ensures auditability.

Proof. The auditability ensures the auditing of the malicious behaviors of parties, especially the semi-honest KC and PV in our framework. On the one hand, since our framework is based on decentralized blockchain Ethereum, the transactions and records are transparent and immutable. On the other hand, the hash of registration state related to the KC and the NIZK proof related to the PV are publicly verifiable and stored on-chain. Therefore, the behaviors of the KC and PV are publicly auditable at any time. □

■ 7 Theoretical Analysis and Evaluation

7.1 Theoretical Analysis

Table 1 shows the comparison between some existing RABE schemes and our scheme. Since our scheme is built upon [5], the parameters are identical to those of [5], except for the decryption cost. Furthermore, our scheme achieves superior parameter complexity compared to other schemes [4,7,8]. Additionally, our outsourced decryption significantly reduces the final decryption cost to a constant level, eliminating the need for pairing operations and requiring only a single group exponentiation operation.

Table 2 shows the comparison of the previous OABE schemes with our scheme in terms of authority, verifiability, exemptibility, fairness,

Table 2 Comparison with OABE Schemes

Schemes	Authority-Free	Verifiability	Exemptibility	Fairness	Auditability	Decryption Cost
[15]	✗	✓	✗	✗	✗	$2\mathbb{E}_G + 2\mathbb{E}_T$
[16, 17]	✗	✓	✗	✗	✗	$\mathbb{E}_T$
[18]	✗	✓	✗	✗	✗	$2\mathbb{E}_G + \mathbb{E}_T$
[27]	✗	✓	✗	✗	✗	$2\mathbb{E}_G + \mathbb{E}_T$
[25]	✗	✓	✓	✓	✗	$\mathbb{E}_T$
Ours	✓	✓	✓	✓	✓	$\mathbb{E}_T$

auditability, and decryption cost. Our scheme is the only one that does not rely on a trusted authority and achieves auditability. Moreover, our scheme achieves the lowest decryption overhead among all the compared schemes. Although all schemes ensure verifiability, only our scheme and [25] guarantee exemptibility and fairness.

7.2 Experimental Evaluation

Our experiment is executed on an Intel(R) Core(TM) i9-12900H CPU and 4 GB RAM running Ubuntu 20.04 LTS 64-bit. The access policy is set as an AND gate of attributes and its size varies from 10 to 100, with a step size of 10.

ORABE Cost. We conduct experiments on our scheme and the RABE scheme of [5] and implement them in Python language within the Charm framework. Our benchmark uses SS512 and SS1024 elliptic curves and the SHA-256 hash function. For each algorithm, we repeat 1,000 times to get the average execution time.

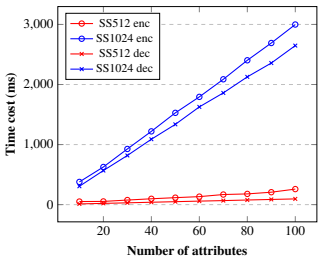


Fig. 5 Encrypt and decrypt of [5]

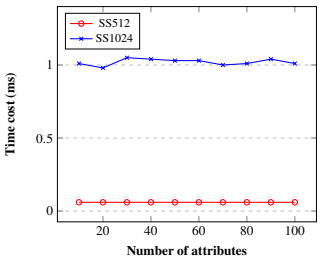


Fig. 6 ORABE **Decrypt**$_{user}$

As shown in Fig 5, the encryption and decryption time cost of the RABE scheme in [5] increases linearly with the number of attributes. Fig 6 demonstrates our scheme has very low and constant decryption time cost, which is 196.5×-1585× on the SS512 curve and 304×-2647× on the SS1024 curve lower than [5] in the number of attributes from 10 to 100. As shown in Fig 7 and Fig 8, the size of the original ciphertext increases linearly with the number of attributes, while the transform ciphertext, which contains only two $\mathbb{G}_T$ elements, has constant size, thereby significantly reducing the storage burden on users.

Fraud Proof Cost. The proving and verifying times of the fraud proof are approximately 0.87ms and 1.49ms on the SS512 curve and 11.81ms and 21.40ms on the SS1024 curve, respectively. These results demonstrate that the fraud proof mechanism is efficient for mobile devices, particularly on the SS512 curve.

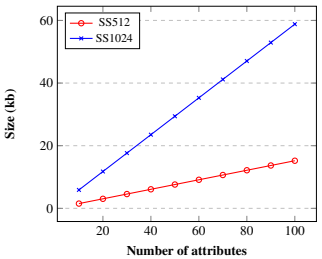


Fig. 7 Original ciphertext

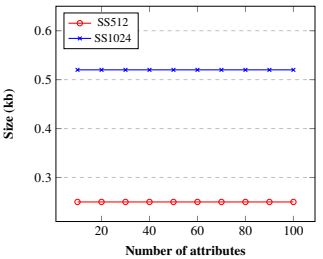


Fig. 8 Transform ciphertext

Gas Usage. We implement our ORABE smart contract described in Section 5.4 in solidity. We deploy it on a locally set up Ethereum test network powered by Hardhat and invoke each function 100 times to get the average gas usage.

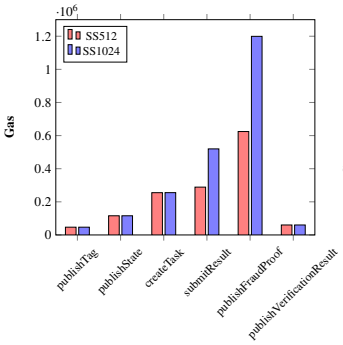


Fig. 9 Gas of our contract

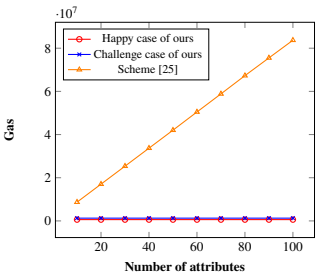


Fig. 10 Gas comparison with [25]

As shown in Fig 9, all functions in our smart contract exhibit constant gas usage. Fig 10 presents a comparison of gas usage across three scenarios: the happy case and challenge case of our scheme, and the scheme proposed by [25]. Note that the gas usage of pairing operations in [25] cannot be tested on native Ethereum due to incompatibility. Therefore, we adopt the experimental result $168164 + k \times 46308$ reported in their work, where k is the number of pairing operations. The results demonstrate that our scheme maintains constant gas usage and achieves significantly lower usage compared to [25].

8 Conclusion

In this paper, we propose ORABE, an auditable registered attribute-based encryption scheme with reliable outsourced decryption based on blockchain. Our scheme achieves verifiability through a verifiable tag mechanism and ensures exemptibility via NIZK proofs. Building upon ORABE, we design a decentralized and auditable data sharing framework without a centralized authority. Additionally, we provide a formal security analysis of the proposed scheme. Finally, we compare the theoretical complexity and functionality of our scheme with existing works and evaluate it on Ethereum to demonstrate feasibility and efficiency. In future work, we plan to explore the applications of our proposed scheme in various innovative scenarios within the metaverse, particularly in data sharing of emerging domains such as SocialFi (Social Finance) and GameFi (Game Finance).

Acknowledgement

This work was supported by National Natural Science Foundation of China (Grant Nos. 62272107 and 62302129), the Project of Shanghai Science and Technology (Grant No. 2024ZX01), Hainan Province Key R&D plan project (No. ZDYF2024GXJS030), the HK RGC General Research Fund (Nos. 16208120 and 16214121), and the Project of the Yiwu Research Institute of Fudan University.

Competing Interest

The authors declare that they have no competing interests or financial conflicts to disclose.

References

[1] Gan W, Ye Z, Wan S, Yu P S. Web 3.0: The future of internet. In: Companion Proceedings of the ACM Web Conference. 2023, 1266–1275

[2] Bethencourt J, Sahai A, Waters B. Ciphertext-policy attribute-based encryption. In: Proceedings of the 2007 IEEE Symposium on Security and Privacy. 2007, 321–334

[3] Lewko A, Waters B. Decentralizing attribute-based encryption. In: Proceedings of the 30th Annual International Conference on the Theory and Applications of Cryptographic Techniques. 2011, 568–588

[4] Hohenberger S, Lu G, Waters B, Wu D J. Registered attribute-based encryption. In: Proceedings of the 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques. 2023, 511–542

[5] Garg R, Lu G, Waters B, Wu D J. Reducing the crs size in registered abe systems. In: Proceedings of the 44th Annual International Cryptology Conference. 2024, 143–177

[6] Freitag C, Waters B, Wu D J. How to use (plain) witness encryption: Registered abe, flexible broadcast, and more. In: Proceedings of the 43rd Annual International Cryptology Conference. 2023, 498–531

[7] Zhu Z, Zhang K, Gong J, Qian H. Registered abe via predicate encodings. In: Proceedings of the 29th International Conference on the Theory and Application of Cryptology and Information Security. 2023, 66–97

[8] Attrapadung N, Tomida J. A modular approach to registered abe for unbounded predicates. In: Proceedings of the 44th Annual International Cryptology Conference. 2024, 280–316

[9] Weng W, Li J, Zhang Y, Lu Y, Shen J, Han J. Efficient registered attribute based access control with same sub-policies in mobile cloud computing. IEEE Transactions on Mobile Computing, 2025

[10] Li J, Chen S, Lu Y, Ning J, Shen J, Zhang Y. Revocable registered attribute-based encryption with user deregistration. IEEE Internet of Things Journal, 2025

[11] Goyal V, Pandey O, Sahai A, Waters B. Attribute-based encryption for fine-grained access control of encrypted data. In: Proceedings of the 13th ACM Conference on Computer and Communications Security. 2006, 89–98

[12] Waters B. Ciphertext-policy attribute-based encryption: An expressive, efficient, and provably secure realization. In: Proceedings of the 14th International Conference on Practice and Theory in Public Key Cryptography. 2011, 53–70

[13] Green M, Hohenberger S, Waters B. Outsourcing the decryption of abe ciphertexts. In: Proceedings of the 20th USENIX Conference on Security. 2011

[14] ElGamal T. A public key cryptosystem and a signature scheme based on discrete logarithms. IEEE Transactions on Information Theory, 1985, 31(4): 469–472

[15] Lai J, Deng R H, Guan C, Weng J. Attribute-based encryption with verifiable outsourced decryption. IEEE Transactions on Information Forensics and Security, 2013, 8(8): 1343–1354

[16] Li J, Huang X, Li J, Chen X, Xiang Y. Securely outsourcing attribute-based encryption with checkability. IEEE Transactions on Parallel and Distributed Systems, 2014, 25(8): 2201–2210

[17] Qin B, Deng R H, Liu S, Ma S. Attribute-based encryption with efficient verifiable outsourced decryption. IEEE Transactions on Information Forensics and Security, 2015, 10(7): 1384–1393

[18] Lin S, Zhang R, Ma H, Wang M. Revisiting attribute-based encryption with verifiable outsourced decryption. IEEE Transactions on Information Forensics and Security, 2015, 10(10): 2119–2130

[19] Miao Y, Li F, Li X, Ning J, Li H, Choo K K R, Deng R H. Verifiable outsourced attribute-based encryption scheme for cloud-assisted mobile e-health system. IEEE Transactions on Dependable and Secure Computing, 2024, 21(4): 1845–1862

[20] Xu J, Wen Q, Li W, Jin Z. Circuit ciphertext-policy attribute-based hybrid encryption with verifiable delegation in cloud computing. IEEE Transactions on Parallel and Distributed Systems, 2016, 27(1): 119–129

[21] Li J, Wang Y, Zhang Y, Han J. Full verifiability for outsourced decryption in attribute based encryption. IEEE Transactions on Services Computing, 2020, 13(3): 478–487

[22] Cui H, Wan Z, Wei X, Nepal S, Yi X. Pay as you decrypt: Decryption outsourcing for functional encryption using blockchain. IEEE Transactions on Information Forensics and Security, 2020, 15: 3227–3238

[23] Nakamoto S. Bitcoin: A peer-to-peer electronic cash system. 2008

[24] Wood G. Ethereum: A secure decentralised generalised transaction ledger. Ethereum project yellow paper, 2014

[25] Ge C, Liu Z, Susilo W, Fang L, Wang H. Attribute-based encryption with reliable outsourced decryption in cloud computing using smart contract. IEEE Transactions on Dependable and Secure Computing, 2024, 21(2): 937–948

[26] Sahai A, Waters B. Fuzzy identity-based encryption. In: Proceedings of the 24th Annual International Conference on the Theory and Applications of Cryptographic Techniques. 2005, 457–473

[27] Ma H, Zhang R, Wan Z, Lu Y, Lin S. Verifiable and exculpable outsourced attribute-based encryption for access control in cloud computing. IEEE Transactions on Dependable and Secure Computing, 2017, 14(6): 679–692

[28] Datta P, Pal T. Registration-based functional encryption. Cryptology

ePrint Archive, 2023, 457
[29] Datta P, Pal T, Yamada S. Registered fe beyond predicates:(attribute-based) linear functions and more. In: Proceedings of the 30th International Conference on the Theory and Application of Cryptology and Information Security. 2024, 65–104
[30] Francati D, Friolo D, Maitra M, Malavolta G, Rahimi A, Venturi D. Registered (inner-product) functional encryption. In: Proceedings of the 29th International Conference on the Theory and Application of Cryptology and Information Security. 2023, 98–133
[31] Chen S, Li J, Zhang Y, Han J. Efficient revocable attribute-based encryption with verifiable data integrity. IEEE Internet of Things Journal, 2024, 11(6): 10441–10451
[32] Fiat A, Shamir A. How to prove yourself: Practical solutions to identification and signature problems. In: Proceedings of the 6th Annual International Cryptology Conference. 1986, 186–194